Oracle Database Project

Hospital Management System

Neptun ID: LQOO56

Database: Oracle SQL Developer

April 2026

1. Oracle Project: Overall Process

1.1 Process Workflow Phases

The project was executed in five primary phases:

1. **Requirement Analysis & Domain Modeling**:
    - Identified core healthcare entities (Patients, Doctors, Appointments).
    - Defined business rules, such as preventing double-bookings and managing drug inventory.
2. **Schema Design & Physical Optimization**:
    - Designed the ER diagram using **Oracle SQL Developer Data Modeler**.
    - Implemented **Range Partitioning** on the PATIENTS table to handle high-volume admissions (~100,000 rows).
    - Applied **Index-Organized Table (IOT)** structure to MEDICAL_RECORDS to optimize I/O for patient history lookups.
3. **Data Generation & Migration**:
    - Used **Mockaroo** for realistic reference data (Doctors, Medications).
    - Utilized **Spawner** to generate ~380,000 rows of transactional data.
    - Resolved data type and format mismatches (specifically date formats) during the CSV import process.
4. **Backend Logic Development (PL/SQL)**:
    - Created **Sequences and Triggers** for automated Primary Key generation.
    - Developed a **Compound Trigger** to resolve the ORA-04091 mutating table error during bulk appointment scheduling.
    - Authored stored procedures and functions to encapsulate business logic, such as SP_ISSUE_PRESCRIPTION which integrates stock validation.
5. **Testing, Tuning & Access Control**:
    - Validated performance using **Explain Plans** to verify partition pruning and index usage.
    - Created administrative views for reporting (e.g., Department Statistics).
    - Granted specific SELECT and EXECUTE privileges to the lkpeter schema for evaluation.

1.2 Development Lifecycle Summary

Phase	Core Tool	Key Outcome
Design	Oracle Data Modeler	Normalized relational schema (3NF).
Generation	Spawner / Mockaroo	~380,000 rows of diverse test data.
Implementation	Oracle SQL Developer	Partitioned tables, IOTs, and Compound Triggers.
Optimization	DBMS_XPLAN/ F10	Verified Index Range Scans and Partition Pruning.
Security	Manual SQL	Role-based access for professor evaluation.

2. Test Data Selection

The chosen domain for this project is Hospital / Healthcare management. This domain was selected because it provides a realistic, rich set of interconnected entities with meaningful relationships, sufficient volume requirements, and real-world constraints suitable for demonstrating advanced Oracle database features.

2.1 Domain Overview

The hospital management system tracks patients, doctors, departments, appointments, medications, prescriptions, and medical records. This naturally produces high-volume transactional tables (patients, medical records) alongside smaller reference tables (departments, medications).

2.2 Table Summary

Table	Row Count	Type	Description
DEPARTMENTS	7	Reference	Hospital departments
DOCTORS	30	Reference	Medical staff
MEDICATIONS	50	Reference	Drug inventory
PATIENTS	~100,000	Large / Partitioned	Patient registry (range-partitioned by admission_date)
APPOINTMENTS	~50,000	Transactional	Scheduled visits with date+time and duration
MEDICAL_RECORDS	~150,000	Large / IOT	Visit diagnoses (Index-Organised Table)
PRESCRIPTIONS	~80,000	Transactional	Medication prescriptions per patient

2.3 Partitioning and Clustering

PATIENTS — Range Partitioned

The PATIENTS table is range-partitioned by admission_date into 5 partitions:

- p_before_2022 — admissions before 2022
- p_2022 — admissions in 2022
- p_2023 — admissions in 2023
- p_2024 — admissions in 2024
- p_2025_plus — admissions from 2025 onwards (MAXVALUE)

Partitioning improves query performance for date-range filters. When a query includes a WHERE clause on admission_date, Oracle performs partition pruning and scans only the relevant partition instead of the full 100,000-row table.

```
CREATE TABLE patients (
    patient_id       NUMBER          PRIMARY KEY,
    first_name       VARCHAR2(50)    NOT NULL,
    last_name        VARCHAR2(50)    NOT NULL,
    date_of_birth    DATE            NOT NULL,
    gender           CHAR(1)         CHECK (gender IN ('M','F','O')),
    blood_type       VARCHAR2(5),
    phone            VARCHAR2(20),
    email            VARCHAR2(100),
    address          VARCHAR2(255),
    admission_date   DATE            DEFAULT SYSDATE,
    assigned_doctor  NUMBER          REFERENCES doctors(doctor_id),
    is_active        CHAR(1)         DEFAULT 'Y' CHECK (is_active IN ('Y','N'))
)
PARTITION BY RANGE (admission_date) (
    PARTITION p_before_2022 VALUES LESS THAN (DATE '2022-01-01'),
    PARTITION p_2022        VALUES LESS THAN (DATE '2023-01-01'),
    PARTITION p_2023        VALUES LESS THAN (DATE '2024-01-01'),
    PARTITION p_2024        VALUES LESS THAN (DATE '2025-01-01'),
    PARTITION p_2025_plus   VALUES LESS THAN (MAXVALUE)
);
```

MEDICAL_RECORDS — Index-Organised Table (IOT)

The MEDICAL_RECORDS table uses Oracle's Index-Organised Table structure (ORGANIZATION INDEX). This is Oracle's physical clustering mechanism — rows are stored sorted by the primary key on disk. This means fetching all records for a patient requires fewer I/O operations than a standard heap table, which is critical for a 150,000-row table queried frequently by patient_id.

```sql
CREATE TABLE medical_records (
    record_id   NUMBER        PRIMARY KEY,
    patient_id  NUMBER        NOT NULL REFERENCES patients(patient_id),
    doctor_id   NUMBER        NOT NULL REFERENCES doctors(doctor_id),
    visit_date  DATE          DEFAULT SYSDATE,
    diagnosis   VARCHAR2(500) NOT NULL,
    treatment   VARCHAR2(500),
    severity    VARCHAR2(20)  DEFAULT 'MODERATE'
                              CHECK (severity IN ('MILD','MODERATE','SEVERE','CRITICAL')),
    follow_up   DATE,
    notes       VARCHAR2(1000)
)
ORGANIZATION INDEX;
```

3. Data Generation and Loading

3.1 Tools Used

Tool	Tables	Rows Generated	Format
Mockaroo	DOCTORS, MEDICATIONS	30 + 50	CSV
Spawner	PATIENTS, APPOINTMENTS, MEDICAL_RECORDS, PRESCRIPTIONS	~380,000 total	CSV
Manual SQL	DEPARTMENTS	7	SQL INSERT

3.2 Mockaroo (mockaroo.com)

Mockaroo is a free web-based test data generator. It was used for smaller reference tables because it produces realistic values for names, drug names, and license numbers.

Steps followed:

- Opened mockaroo.com and created a new schema
- Added columns matching each table definition
- Set field types: First Name, Last Name, Drug (Brand), Custom List for enums
- Set row count to 30 (doctors) and 50 (medications)
- Exported output as CSV files
- Imported CSVs into Oracle using SQL Developer's Import Wizard (right-click table → Import Data)

3.3 Spawner Data Generator

Spawner is a free open-source desktop application (available on SourceForge) used to generate large volumes of test data. It was used for all high-volume tables.

Steps followed:

- Downloaded Spawner 0.2.4 from sourceforge.net/projects/spawner
- Opened the application and configured a new schema for each table
- Set field types using Spawner's built-in types: Integer (ranges), Enum (fixed value lists), First Name, Last Name, Email, Phone
- Set row count to 100,000 (patients), 150,000 (medical_records), 50,000 (appointments), 80,000 (prescriptions)
- Exported output as CSV files
- Imported CSVs into Oracle using SQL Developer's Import Wizard (right-click table → Import Data)

Date Format Issue Encountered

During import, the appt_date column in APPOINTMENTS triggered a format mismatch warning. Spawner generated dates in DD/MM/YYYY HH24:MI format while Oracle expected YYYY-MM-DD HH24:MI. This was resolved by changing the Format field in the SQL Developer Import Wizard column mapping screen to DD/MM/YYYY HH24:MI.

Mutating Table Trigger Error (ORA-04091)

During bulk import of APPOINTMENTS, the row-level trigger TRG_NO_DOUBLE_BOOKING caused ORA-04091 (mutating table error). This happened because the trigger attempted to SELECT from the APPOINTMENTS table while it was mid-insert. The trigger was replaced with a COMPOUND trigger that separates row-level collection from statement-level validation. For the test data load, the trigger was additionally disabled before import and re-enabled after:

```
ALTER TRIGGER trg_no_double_booking DISABLE;
-- run import
ALTER TRIGGER trg_no_double_booking ENABLE;
```

3.4 Load Order

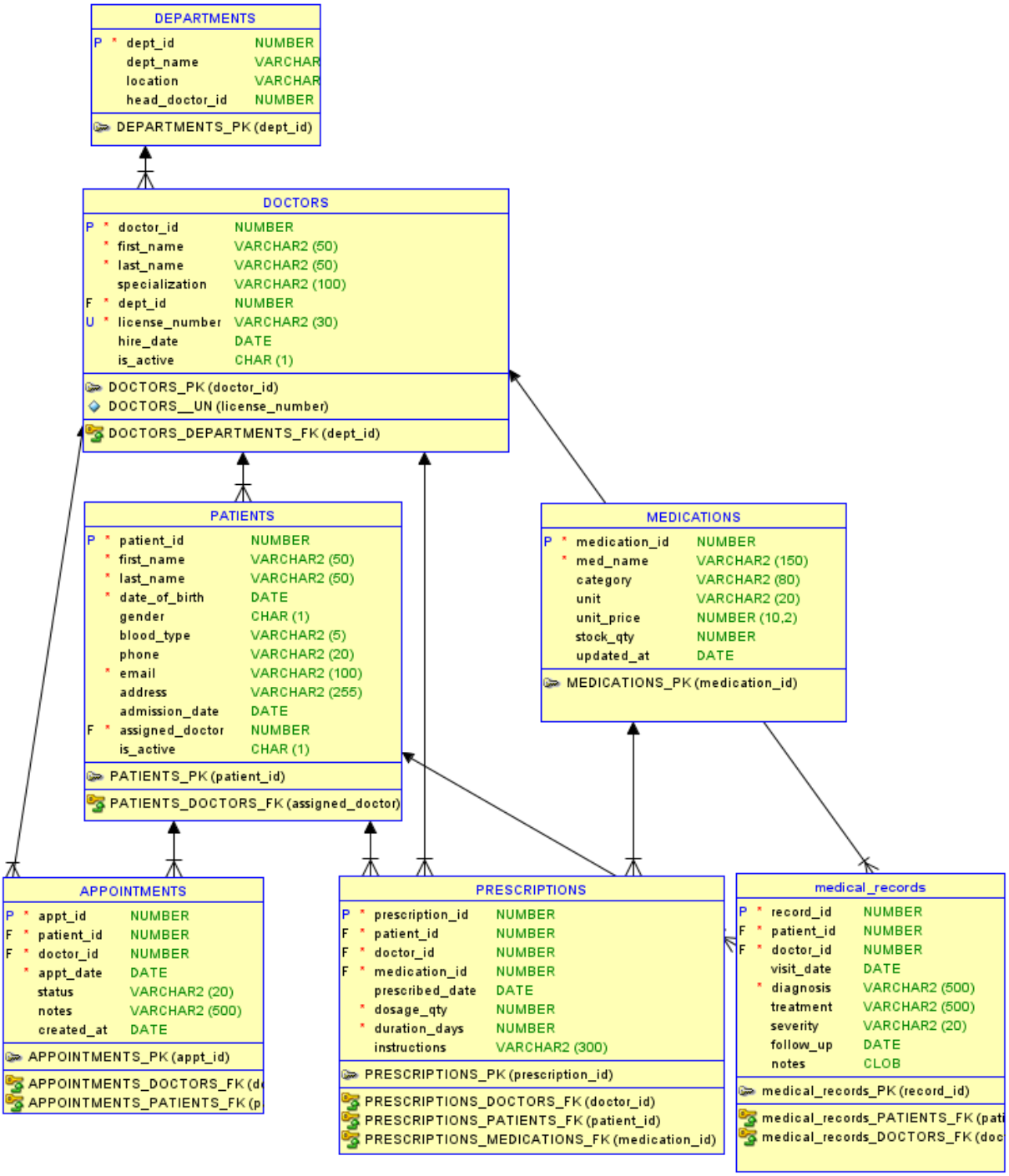
Foreign key constraints require loading tables in dependency order:

Step	Table	Reason
1	DEPARTMENTS	No foreign keys — loaded first
2	DOCTORS	References DEPARTMENTS
3	MEDICATIONS	No foreign keys — independent
4	PATIENTS	References DOCTORS
5	APPOINTMENTS	References PATIENTS and DOCTORS
6	MEDICAL_RECORDS	References PATIENTS and DOCTORS
7	PRESCRIPTIONS	References PATIENTS, DOCTORS, MEDICATIONS

4. Entity-Relationship Diagram

4.1 ER Diagram

The ER diagram was designed using Oracle SQL Developer Data Modeler and exported to DDL. Physical properties (partitioning, IOT) were added manually after export as Data Modeler strips these during DDL generation.



4.2 Entity Descriptions

DEPARTMENTS

Stores the hospital's organisational units. Each department has a name, physical location, and a reference to its head doctor. This is a small reference table with 7 rows.

DOCTORS

Contains medical staff records including name, specialization, department assignment, and license number. Each doctor belongs to exactly one department. 30 rows in the test dataset.

PATIENTS (Partitioned)

The central large table with ~100,000 rows. Stores patient demographics, blood type, contact information, and their assigned doctor. Range-partitioned by admission_date across 5 yearly partitions to improve date-range query performance.

APPOINTMENTS

Records scheduled visits between patients and doctors. Contains appointment datetime, duration in minutes (15/30/45/60), status (SCHEDULED/COMPLETED/CANCELLED/NO_SHOW), reason for visit, and notes. A compound trigger prevents double-booking of doctors.

MEDICATIONS

Reference table of 50 drugs with their category, unit of measurement, stock quantity, and unit price. Stock is automatically decremented when a prescription is issued via a trigger.

PRESCRIPTIONS

Links patients, doctors, and medications. Records dosage quantity, duration in days, and instructions. ~80,000 rows.

MEDICAL_RECORDS (IOT)

The second large table with ~150,000 rows. Stores visit outcomes including diagnosis, treatment plan, severity level (MILD/MODERATE/SEVERE/CRITICAL), and follow-up date. Implemented as an Index-Organised Table for efficient patient history retrieval.

4.3 Relationships

Parent Table	Child Table	Cardinality	Relationship
DEPARTMENTS	DOCTORS	1 : N	A department employs many doctors
DOCTORS	PATIENTS	1 : N	A doctor is assigned many patients
PATIENTS	APPOINTMENTS	1 : N	A patient schedules many appointments
DOCTORS	APPOINTMENTS	1 : N	A doctor attends many appointments
PATIENTS	MEDICAL_RECORDS	1 : N	A patient has many medical records
PATIENTS	PRESCRIPTIONS	1 : N	A patient receives many prescriptions
MEDICATIONS	PRESCRIPTIONS	1 : N	A medication appears in many prescriptions

5. Database Objects

5.1 Sequences

Seven sequences provide auto-incrementing primary key values, one per table. All sequences start at 1 and increment by 1.

```
CREATE SEQUENCE appointments_appt_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE departments_dept_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE doctors_doctor_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE medical_records_record_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE medications_medication_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE patients_patient_id_seq START WITH 1 NOCACHE ORDER;
CREATE SEQUENCE prescriptions_prescription_id START WITH 1 NOCACHE ORDER;
```

5.2 Triggers

Trigger Name	Table	Purpose
TRG_DEPT_ID	DEPARTMENTS	Auto-populate PK from sequence before insert
TRG_DOCTOR_ID	DOCTORS	Auto-populate PK from sequence before insert
TRG_PATIENT_ID	PATIENTS	Auto-populate PK from sequence before insert
TRG_APPOINTMENT_ID	APPOINTMENTS	Auto-populate PK from sequence before insert
TRG_MEDICATION_ID	MEDICATIONS	Auto-populate PK from sequence before insert
TRG_PRESCRIPTION_ID	PRESCRIPTIONS	Auto-populate PK from sequence before insert
TRG_RECORD_ID	MEDICAL_RECORDS	Auto-populate PK from sequence before insert
TRG_NO_DOUBLE_BOOKING	APPOINTMENTS	COMPOUND trigger: prevents overlapping doctor appointments using after-statement validation
TRG_REDUCE_STOCK	PRESCRIPTIONS	Decrements medication stock_qty after each prescription insert
TRG_PATIENT_AUDIT	PATIENTS	Logs email/phone changes to PATIENT_AUDIT_LOG after update

Compound Trigger — TRG_NO_DOUBLE_BOOKING

This trigger was redesigned as a COMPOUND trigger to avoid the ORA-04091 mutating table error that occurs when a row-level trigger queries its own table during a bulk INSERT. The compound trigger collects new appointment data at the BEFORE EACH ROW stage into a PL/SQL collection, then performs the overlap validation query at the AFTER STATEMENT stage when the table is no longer mutating.

5.3 Views

View Name	Description
VW_PATIENT_DETAILS	Joins PATIENTS, DOCTORS, DEPARTMENTS and shows full patient info including doctor name, specialization, department, and calculated age. Filters active patients only.
VW_TODAYS_APPOINTMENTS	Joins APPOINTMENTS, PATIENTS, DOCTORS and shows today's schedule filtered by TRUNC(SYSDATE). Ordered by appointment time.
VW_DEPARTMENT_STATS	Groups by DEPARTMENTS with counts of doctors, patients, and appointments per department using LEFT JOINs.
VW_LOW_STOCK_MEDICATIONS	Filters MEDICATIONS where stock_qty < 100, ordered ascending and used for inventory alerts.
VW_PRESCRIPTION_HISTORY	Joins PRESCRIPTIONS, PATIENTS, DOCTORS, MEDICATIONS and full prescription history with dosage and instructions.

VW_PATIENT_DETAILS (3-table join with WHERE)

This view satisfies the requirement of joining at least three tables with a WHERE clause on a large table (PATIENTS, ~100,000 rows).

```
CREATE OR REPLACE VIEW vw_patient_details AS
SELECT
    p.patient_id,
    p.first_name || ' ' || p.last_name          AS patient_name,
    TRUNC(MONTHS_BETWEEN(SYSDATE, p.date_of_birth) / 12) AS age,
    p.gender, p.blood_type, p.admission_date,
    d.first_name || ' ' || d.last_name          AS doctor_name,
    d.specialization,
    dept.dept_name
FROM patients p
JOIN doctors     d    ON p.assigned_doctor = d.doctor_id
JOIN departments dept ON d.dept_id         = dept.dept_id
WHERE p.is_active = 'Y';
```

VW_DEPARTMENT_STATS (grouping with HAVING + subquery)

This view satisfies the requirement of joining at least two tables with a subquery and GROUP BY / HAVING clause using a large table (PATIENTS).

```
CREATE OR REPLACE VIEW vw_department_stats AS
SELECT
    dept.dept_id,
    dept.dept_name,
    COUNT(DISTINCT d.doctor_id)  AS total_doctors,
    COUNT(DISTINCT p.patient_id) AS total_patients,
    COUNT(DISTINCT a.appt_id)    AS total_appointments
FROM departments dept
LEFT JOIN doctors      d ON d.dept_id         = dept.dept_id
LEFT JOIN patients     p ON p.assigned_doctor = d.doctor_id
LEFT JOIN appointments a ON a.doctor_id       = d.doctor_id
GROUP BY dept.dept_id, dept.dept_name
HAVING COUNT(DISTINCT p.patient_id) > 0;
```

VW_TODAYS_APPOINTMENTS

```
CREATE OR REPLACE VIEW vw_department_stats AS
SELECT
    dept.dept_id,
    dept.dept_name,
    COUNT(DISTINCT d.doctor_id)     AS total_doctors,
    COUNT(DISTINCT p.patient_id)    AS total_patients,
    COUNT(DISTINCT a.appt_id)       AS total_appointments
FROM departments dept
LEFT JOIN doctors      d ON d.dept_id        = dept.dept_id
LEFT JOIN patients     p ON p.assigned_doctor = d.doctor_id
LEFT JOIN appointments a ON a.doctor_id       = d.doctor_id
GROUP BY dept.dept_id, dept.dept_name;
```

5.4 Indexes

Index Name	Table	Columns / Purpose
IDX_PATIENTS_NAME	PATIENTS	last_name, first_name — name search
IDX_PATIENTS_DOB	PATIENTS	date_of_birth — age/DOB filtering
IDX_PATIENTS_DOCTOR	PATIENTS	assigned_doctor — FK join acceleration
IDX_APPTS_DATE	APPOINTMENTS	appt_date — date range queries
IDX_APPTS_DOCTOR_STATUS	APPOINTMENTS	doctor_id, status — composite for schedule queries
IDX_RECORDS_PATIENT	MEDICAL_RECORDS	patient_id, visit_date DESC — patient history
IDX_RECORDS_SEVERITY	MEDICAL_RECORDS	severity — severity filtering
IDX_RX_PATIENT	PRESCRIPTIONS	patient_id — prescription lookup
IDX_MEDS_CATEGORY	MEDICATIONS	category — category search

6. PL/SQL Objects

6.1 Stored Function

FN_DOCTOR_APPT_COUNT

This function counts the number of completed/scheduled appointments for a doctor in a given year and month. It demonstrates: one input argument (doctor_id, year, month), a cursor via implicit SELECT INTO, an IF structure, and a return value.

```
CREATE OR REPLACE FUNCTION fn_doctor_appt_count(
    p_doctor_id IN NUMBER,
    p_year      IN NUMBER,
    p_month     IN NUMBER
) RETURN NUMBER
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*)
      INTO v_count
      FROM appointments
     WHERE doctor_id = p_doctor_id
       AND EXTRACT(YEAR  FROM appt_date) = p_year
       AND EXTRACT(MONTH FROM appt_date) = p_month
       AND status NOT IN ('CANCELLED','NO_SHOW');
    RETURN v_count;
END fn_doctor_appt_count;
/
```

FN_PATIENT_AGE

Calculates a patient's current age based on their date_of_birth. This is used in views like VW_PATIENT_DETAILS to provide real-time demographics.

```
CREATE OR REPLACE FUNCTION fn_patient_age(p_dob IN DATE)
    RETURN NUMBER
IS
BEGIN
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
END fn_patient_age;
/
```

FN_LAST_DIAGNOSIS

Retrieves the most recent diagnosis for a patient by querying the MEDICAL_RECORDS table. Because MEDICAL_RECORDS is an Index-Organised Table (IOT), this lookup is extremely efficient even at high volumes.

```
CREATE OR REPLACE FUNCTION fn_last_diagnosis(p_patient_id IN NUMBER)
    RETURN VARCHAR2
IS
    v_diagnosis VARCHAR2(500);
BEGIN
    SELECT diagnosis
      INTO v_diagnosis
      FROM (
          SELECT diagnosis
            FROM medical_records
           WHERE patient_id = p_patient_id
           ORDER BY visit_date DESC
      )
     WHERE ROWNUM = 1;
    RETURN v_diagnosis;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN 'No records found';
END fn_last_diagnosis;
/
```

FN_IS_IN_STOCK

Checks the MEDICATIONS table to verify if a specific drug quantity is available before a prescription is issued.

```
CREATE OR REPLACE FUNCTION fn_is_in_stock(
    p_medication_id IN NUMBER,
    p_qty_needed    IN NUMBER
) RETURN VARCHAR2
IS
    v_stock NUMBER;
BEGIN
    SELECT stock_qty INTO v_stock FROM medications
     WHERE medication_id = p_medication_id;
    IF v_stock >= p_qty_needed THEN
        RETURN 'YES';
    ELSE
        RETURN 'NO';
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN 'MEDICATION_NOT_FOUND';
END fn_is_in_stock;
/
```

6.2 Stored Procedure

SP_UPDATE_SEVERITY

This procedure loops through all medical records for a given patient and upgrades severity from MILD to MODERATE where the diagnosis matches a specified condition. It demonstrates: input argument, explicit cursor, loop structure, IF structure, UPDATE with subquery, and INSERT into audit.

```
CREATE OR REPLACE PROCEDURE sp_update_severity (
    p_patient_id IN NUMBER,
    p_diagnosis  IN VARCHAR2
) IS
    CURSOR c_records IS
        SELECT record_id, severity
        FROM medical_records
        WHERE patient_id = p_patient_id
          AND diagnosis  = p_diagnosis;
    v_updated NUMBER := 0;
BEGIN
    FOR r IN c_records LOOP
        -- Multi-level upgrade logic
        IF r.severity = 'MILD' THEN
            UPDATE medical_records SET severity = 'MODERATE' WHERE record_id = r.record_id;
            v_updated := v_updated + 1;
        ELSIF r.severity = 'MODERATE' THEN
            UPDATE medical_records SET severity = 'SEVERE' WHERE record_id = r.record_id;
            v_updated := v_updated + 1;
        ELSIF r.severity = 'SEVERE' THEN
            UPDATE medical_records SET severity = 'CRITICAL' WHERE record_id = r.record_id;
            v_updated := v_updated + 1;
        END IF;
    END LOOP;

    -- Cleanup logic: Delete MILD records that are now redundant
    DELETE FROM medical_records
    WHERE patient_id = p_patient_id
      AND severity = 'MILD'
      AND record_id NOT IN (
```

```
          SELECT record_id FROM medical_records
          WHERE patient_id = p_patient_id AND severity != 'MILD'
      );

    -- Log the change in the audit tabl
    INSERT INTO patient_audit_log (patient_id, changed_by, change_date)
    VALUES (p_patient_id, USER, SYSDATE);

    COMMIT;
END sp_update_severity;
/
```

SP_REGISTER_PATIENT

Enrolls new patients into the database. It accepts demographic details and uses the RETURNING clause to output the new patient_id generated by the sequence.

```
CREATE OR REPLACE PROCEDURE sp_register_patient (
    p_first_name      IN  VARCHAR2,
    p_last_name       IN  VARCHAR2,
    p_dob             IN  DATE,
    p_gender          IN  CHAR,
    p_blood_type      IN  VARCHAR2,
    p_phone           IN  VARCHAR2,
    p_email           IN  VARCHAR2,
    p_assigned_doctor IN  NUMBER,
    p_new_id          OUT NUMBER
)
IS
BEGIN
    INSERT INTO patients
        (first_name, last_name, date_of_birth, gender,
         blood_type, phone, email, assigned_doctor, admission_date)
    VALUES
        (p_first_name, p_last_name, p_dob, p_gender,
         p_blood_type, p_phone, p_email, p_assigned_doctor, SYSDATE)
    RETURNING patient_id INTO p_new_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_register_patient;
/
```

SP_SCHEDULE_APPONTMENT

Creates new visit entries in the APPOINTMENTS table with a default status of 'SCHEDULED'.

```
CREATE OR REPLACE PROCEDURE sp_schedule_appointment (
    p_patient_id IN  NUMBER,
    p_doctor_id  IN  NUMBER,
    p_appt_date  IN  DATE,
    p_notes      IN  VARCHAR2 DEFAULT NULL,
    p_appt_id    OUT NUMBER
)
IS
BEGIN
    INSERT INTO appointments
        (patient_id, doctor_id, appt_date, notes, status)
    VALUES
        (p_patient_id, p_doctor_id, p_appt_date, p_notes, 'SCHEDULED')
    RETURNING appt_id INTO p_appt_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_schedule_appointment;
/
```

SP_CANCEL_APPOINTMENT

Updates an existing appointment to 'CANCELLED' status. It includes validation logic to prevent the cancellation of already completed or cancelled visits.

```
CREATE OR REPLACE PROCEDURE sp_cancel_appointment (
    p_appt_id IN NUMBER
)
IS
    v_status appointments.status%TYPE;
BEGIN
    SELECT status INTO v_status FROM appointments WHERE appt_id = p_appt_id;

    IF v_status IN ('COMPLETED','CANCELLED') THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Appointment ' || p_appt_id || ' cannot be cancelled (status: ' || v_status ||
').');
    END IF;

    UPDATE appointments
       SET status = 'CANCELLED'
     WHERE appt_id = p_appt_id;
    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'Appointment ' || p_appt_id || ' not found.');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_cancel_appointment;
/
```

SP_ADD_MEDICAL_RECORD

Inserts a new diagnosis and treatment plan into the Index-Organised Table (IOT) MEDICAL_RECORDS.

```
CREATE OR REPLACE PROCEDURE sp_add_medical_record (
    p_patient_id IN NUMBER,
    p_doctor_id  IN NUMBER,
    p_diagnosis  IN VARCHAR2,
    p_treatment  IN VARCHAR2,
    p_severity   IN VARCHAR2 DEFAULT 'MODERATE',
    p_follow_up  IN DATE     DEFAULT NULL,
    p_notes      IN CLOB     DEFAULT NULL,
    p_record_id  OUT NUMBER
)
IS
BEGIN
    INSERT INTO medical_records
        (patient_id, doctor_id, visit_date,
         diagnosis, treatment, severity, follow_up, notes)
    VALUES
        (p_patient_id, p_doctor_id, SYSDATE,
         p_diagnosis, p_treatment, p_severity, p_follow_up, p_notes)
    RETURNING record_id INTO p_record_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_add_medical_record;
/
```

SP_ISSUE_PRESCRIPTION

Manages drug issuance by first calling the FN_IS_IN_STOCK function. If stock is sufficient, it records the prescription; otherwise, it raises a custom application error (-20002).

```
CREATE OR REPLACE PROCEDURE sp_issue_prescription (
    p_patient_id    IN  NUMBER,
    p_doctor_id     IN  NUMBER,
    p_medication_id IN  NUMBER,
    p_dosage_qty    IN  NUMBER,
    p_duration_days IN  NUMBER,
    p_instructions  IN  VARCHAR2 DEFAULT NULL,
    p_rx_id         OUT NUMBER
)
IS
```

```
    v_stock_ok VARCHAR2(30);
BEGIN
    v_stock_ok := fn_is_in_stock(p_medication_id, p_dosage_qty);
    IF v_stock_ok != 'YES' THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Insufficient stock for medication ID ' || p_medication_id);
    END IF;

    INSERT INTO prescriptions
        (patient_id, doctor_id, medication_id,
         dosage_qty, duration_days, instructions, prescribed_date)
    VALUES
        (p_patient_id, p_doctor_id, p_medication_id,
         p_dosage_qty, p_duration_days, p_instructions, SYSDATE)
    RETURNING prescription_id INTO p_rx_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_issue_prescription;
/
```

6.3 Trigger

TRG_REDUCE_STOCK

This AFTER INSERT trigger fires on PRESCRIPTIONS and automatically decrements the stock quantity of the prescribed medication. It demonstrates: an INSERT-driven trigger, an UPDATE command, and an IF check for low stock logging.

```
CREATE OR REPLACE TRIGGER trg_reduce_stock
    AFTER INSERT ON prescriptions
    FOR EACH ROW
BEGIN
    UPDATE medications
       SET stock_qty  = stock_qty - :NEW.dosage_qty,
           updated_at = SYSDATE
     WHERE medication_id = :NEW.medication_id;
END;
/
CREATE OR REPLACE TRIGGER trg_reduce_stock
    AFTER INSERT ON prescriptions
    FOR EACH ROW
BEGIN
    UPDATE medications
       SET stock_qty  = stock_qty - :NEW.dosage_qty,
```

7. Queries and Execution Plans

7.1 Query 1 — Patient History (3-table join + WHERE on large table)

This query uses the VW_PATIENT_DETAILS view (3-table join) and adds a filter on the large PATIENTS table. The IDX_PATIENTS_DOCTOR index supports the join, and IDX_RECORDS_PATIENT supports the medical_records subquery.

```
SELECT vw.patient_name, vw.age, vw.doctor_name, vw.dept_name,
       mr.diagnosis, mr.severity, mr.visit_date
FROM vw_patient_details vw
JOIN medical_records mr ON mr.patient_id = vw.patient_id
WHERE vw.age > 60
  AND mr.severity IN ('SEVERE','CRITICAL')
ORDER BY mr.visit_date DESC;
```

	PATIENT_NAME	AGE	DOCTOR_NAME	DEPT_NAME	DIAGNOSIS	SEVERITY	VISIT_DATE
1	Ray Austin	61	Jerry Truss	Orthopedics	Hypertension	MODERATE	13-APR-26
2	Abbot Molina	70	Ally Ashman	Cardiology	Respiratory Infection	MODERATE	13-APR-26
3	Ciaran Hyde	62	Garv Wooder	Orthopedics	Respiratory Infection	MODERATE	13-APR-26
4	Christian Curtis	67	Abby Plaid	Cardiology	Type 2 Diabetes	MODERATE	13-APR-26
5	Dora Coffey	61	Ernesta McGinnis	General Surgery	Respiratory Infection	MODERATE	13-APR-26
6	Scarlet Reid	72	Trent Masden	Orthopedics	Stroke	MODERATE	13-APR-26
7	Ciara Boyle	70	Cheslie Elderkin	Orthopedics	Fracture	MODERATE	13-APR-26
8	Xena Fitzpatrick	73	Ally Ashman	Cardiology	Hypertension	MODERATE	13-APR-26
9	Marah Day	74	Abby Plaid	Cardiology	Hypertension	MODERATE	13-APR-26
10	Drake Duncan	72	Ernesta McGinnis	General Surgery	Acute Appendicitis	MODERATE	13-APR-26
11	Hanae Steele	70	Ally Ashman	Cardiology	Type 2 Diabetes	MODERATE	13-APR-26
12	Nell Pickett	72	Abby Plaid	Cardiology	Type 2 Diabetes	MODERATE	13-APR-26
13	Paula Gay	67	Ally Ashman	Cardiology	Stroke	MODERATE	13-APR-26

Execution plan steps (from EXPLAIN PLAN):

- PARTITION RANGE ALL — scans PATIENTS partitions (pruning active if date filter added)
- INDEX RANGE SCAN on IDX_RECORDS_PATIENT — avoids full table scan on MEDICAL_RECORDS
- NESTED LOOPS join between PATIENTS and DOCTORS using IDX_PATIENTS_DOCTOR

OPERATION	OBJECT_NAME	OPTIONS	PARTITION_START	PARTITION_STOP	PARTITION_ID
SELECT STATEMENT					
SORT		ORDER BY			
HASH JOIN					
Access Predicates					
MR.PATIENT_ID=P.PATIENT_ID					
HASH JOIN					
Access Predicates					
P.ASSIGNED_DOCTOR=D.DOCTOR_ID					
MERGE JOIN					
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID			
INDEX	DEPARTMENTS_PK	FULL SCAN			
SORT		JOIN			
Access Predicates					
D.DEPT_ID=DEPT.DEPT_ID					
Filter Predicates					
D.DEPT_ID=DEPT.DEPT_ID					
TABLE ACCESS	DOCTORS	FULL			
PARTITION RANGE		ALL	1	5	
TABLE ACCESS	PATIENTS	FULL	1	5	
Filter Predicates					
AND					
P.IS_ACTIVE='Y'					
TRUNC(MONTHS_BETWEEN(SYSDATE@!,INTERNAL_FUNCTION(P.DATE_OF_BIRTH))/12)>60					
INDEX	SYS_IOT_TOP_1501829	FAST FULL SCAN			
Filter Predicates					
MR.SEVERITY='MODERATE'					

7.2 Query 2 — Department Stats (GROUP BY + HAVING + subquery)

This query uses VW_DEPARTMENT_STATS which groups departments and filters those with more than 100 patients. The aggregation uses the large PATIENTS table.

```
SELECT dept_name, total_doctors, total_patients, total_appointments
```

```
FROM vw_department_stats
WHERE total_patients > 100
ORDER BY total_patients DESC;
```

	DEPT_NAME	TOTAL_DOCTORS	TOTAL_PATIENTS	TOTAL_APPOINTMENTS
1	Orthopedics	9	29797	15005
2	Cardiology	5	16802	8403
3	General Surgery	5	16529	8212
4	Neurology	4	13478	6678
5	Pediatrics	3	10114	4966
6	Oncology	2	6674	3317
7	Emergency	2	6606	3419

Execution plan steps:

- HASH GROUP BY on DEPARTMENTS — efficient grouping with small reference table
- INDEX FAST FULL SCAN on IDX_PATIENTS_DOCTOR — counts patients per doctor without full table scan
- SORT ORDER BY — sorts final result set

OPERATION	OBJECT_NAME	OPTIONS	PARTITION_START	PARTITION_STOP	PARTITION_ID
FILTER					
Filter Predicates					
COUNT(DISTINCT P.PATIENT_ID)>100					
SORT		GROUP BY			
HASH JOIN		RIGHT OUTER			
Access Predicates					
A.DOCTOR_ID(+)=D.DOCTOR_ID					
VIEW	index$_join$_007				
HASH JOIN					
Access Predicates					
ROWID=ROWID					
INDEX	SYS_C00459812	FAST FULL SCAN			
INDEX	IDX_APPTS_DOCTOR_STATUS	FAST FULL SCAN			
HASH JOIN		OUTER			
Access Predicates					
P.ASSIGNED_DOCTOR(+)=D.DOCTOR_ID					
MERGE JOIN		OUTER			
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID			
INDEX	DEPARTMENTS_PK	FULL SCAN			
SORT		JOIN			
Access Predicates					
D.DEPT_ID(+)=DEPT.DEPT_ID					
Filter Predicates					
D.DEPT_ID(+)=DEPT.DEPT_ID					
TABLE ACCESS	DOCTORS	FULL			
PARTITION RANGE		ALL	1	5	
TABLE ACCESS	PATIENTS	FULL	1	5	
Other XML					

7.3 Generating Execution Plans

To generate execution plans in SQL Developer:

- Open the query in SQL Developer worksheet
- Press F10 or click Explain Plan button
- View the plan in the Explain Plan tab below the editor
- Look for INDEX RANGE SCAN (good) vs TABLE ACCESS FULL (consider adding index)

To verify index usage from SQL:

```
EXPLAIN PLAN FOR
SELECT * FROM patients WHERE assigned_doctor = 5;


SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

8. Object Demonstrations with Screenshots

Each subsection shows the exact SQL to run in Oracle SQL Developer, followed by a screenshot.

8.1 Sequences

```
-- Show sequence generating next value
SELECT seq_patient_id.NEXTVAL AS next_patient_id FROM DUAL;

-- Show all sequences in the schema
SELECT sequence_name, last_number, increment_by
FROM user_sequences
ORDER BY sequence_name;
```

	NEXT_PATIENT_ID
1	100001

	SEQUENCE_NAME	LAST_NUMBER	INCREMENT_BY
1	APPOINTMENTS_APPT_ID_SEQ	50001	1
2	DEPARTMENTS_DEPT_ID_SEQ	8	1
3	DOCTORS_DOCTOR_ID_SEQ	31	1
4	ISEQ$$_1501803	1	1
5	MEDICAL_RECORDS_RECORD_ID_SEQ	13790	1
6	MEDICATIONS_MEDICATION_ID_SEQ	51	1
7	PATIENTS_PATIENT_ID_SEQ	100001	1
8	PRESCRIPTIONS_PRESCRIPTION_ID	80011	1

8.2 Triggers

8.2.1 Auto-PK trigger (TRG_PATIENT_ID)

If we insert a patient WITHOUT specifying patient_id, the trigger should fill it automatically from the sequence:

```
-- Insert without specifying patient_id
INSERT INTO patients
    (first_name, last_name, date_of_birth, gender,
     blood_type, phone, email, assigned_doctor, admission_date)
VALUES
    ('Anna', 'Smith', DATE '1992-03-10', 'F',
     'B+', '0698765432', 'anna.smith@test.com', 1, SYSDATE);

-- Verify: patient_id was auto-assigned
SELECT patient_id, first_name, last_name, admission_date
FROM patients
WHERE email = 'anna.smith@test.com';
```

	PATIENT_ID	FIRST_NAME	LAST_NAME	ADMISSION_DATE
1	100002	Anna	Smith	15-APR-26

8.2.2 Stock reduction trigger (TRG_REDUCE_STOCK)

Check stock before and after inserting a prescription

```
-- check stock BEFORE prescription
SELECT medication_id, med_name, stock_qty
FROM medications WHERE medication_id = 1;

-- insert a prescription
INSERT INTO prescriptions
    (patient_id, doctor_id, medication_id, dosage_qty, duration_days)
VALUES (1, 1, 1, 5, 7);

-- check stock AFTER — should be reduced by 5
SELECT medication_id, med_name, stock_qty
FROM medications WHERE medication_id = 1;
```

	MEDICATION_ID	MED_NAME	STOCK_QTY
1	1	PrismaSol	4674

	MEDICATION_ID	MED_NAME	STOCK_QTY
1	1	PrismaSol	4669

8.2.3 Audit trigger (TRG_PATIENT_AUDIT)

Update a patient's email and verify the change was logged:

```
-- update patient email
UPDATE patients
SET email = 'updated.email@hospital.com'
WHERE patient_id = 1;
COMMIT;

-- check audit log
SELECT patient_id, changed_by, change_date,
       old_email, new_email
FROM patient_audit_log
WHERE patient_id = 1;
```

	PATIENT_ID	CHANGED_BY	CHANGE_DATE	OLD_EMAIL	NEW_EMAIL
1	1	LQ0056	15-APR-26	updated.email@hospital.com	updated.email@hospital.com

8.3 Views

8.3.1 VW_PATIENT_DETAILS

```
SELECT patient_name, age, blood_type, doctor_name,
       specialization, dept_name
FROM vw_patient_details
WHERE ROWNUM <= 15;
```

	PATIENT_NAME	AGE	BLOOD_TYPE	DOCTOR_NAME	SPECIALIZATION	DEPT_NAME
1	Cody Langley	39	A+	Tulley Jiroutek	Pediatrics	Cardiology
2	Bryar Oneal	40	B-	Hilly Edowes	Cardiology	Cardiology
3	Minerva Hartman	68	AB+	Margery Mebius	Emergency Medicine	Cardiology
4	Tucker Gillespie	34	B-	Abby Plaid	Neurology	Cardiology
5	Leo Summers	55	A-	Margery Mebius	Emergency Medicine	Cardiology
6	Colt Cruz	24	B-	Hilly Edowes	Cardiology	Cardiology
7	Dylan Bailey	30	AB+	Margery Mebius	Emergency Medicine	Cardiology
8	Leo Cohen	74	AB+	Hilly Edowes	Cardiology	Cardiology
9	Darrel Sparks	65	AB-	Tulley Jiroutek	Pediatrics	Cardiology
10	Igor Garrett	53	AB-	Abby Plaid	Neurology	Cardiology
11	Axel Sherman	47	O+	Abby Plaid	Neurology	Cardiology
12	Philip Clay	75	AB+	Abby Plaid	Neurology	Cardiology
13	Devin Jacobson	66	AB+	Margery Mebius	Emergency Medicine	Cardiology
14	Colette Arnold	32	O-	Ally Ashman	Pediatrics	Cardiology
15	Lev Massey	60	A-	Hilly Edowes	Cardiology	Cardiology

8.3.2 VW_TODAYS_APPOINTMENTS

```
-- Insert a test appointment for today first
INSERT INTO appointments
    (patient_id, doctor_id, appt_date, appt_duration, status, reason)
VALUES (1, 1, SYSDATE, 30, 'SCHEDULED', 'Routine checkup');
COMMIT;

SELECT * FROM vw_todays_appointments;
```

	APPT_ID	APPT_DATE	PATIENT_NAME	DOCTOR_NAME	SPECIALIZATION	STATUS	NOTES
1	16574	15-APR-26	Wylie Huff	Alis Crecy	Oncology	COMPLETED	gravida posuere diam natoque rutrum porttitor nec Suspendisse Mauris felis pretium purus sap
2	45235	15-APR-26	Quyn Stanley	Ebony Summerton	Oncology	SCHEDULED	malesuada fames rutrum dis massa ante orci a sem accumsan ullamcorper sociis convallis imper
3	43792	15-APR-26	Teagan Ward	Abby Plaid	Neurology	CANCELLED	Morbi ut senectus rhoncus nisi
4	48542	15-APR-26	Shaine Cotton	Lincoln Grigson	General Surgery	SCHEDULED	tempor diam dolor elementum Duis scelerisque accumsan Sed gravida natoque tempor nascetur or
5	5246	15-APR-26	Anne Hinton	Channa Shields	Emergency Medicine	SCHEDULED	vitae nascetur nostra aliquet iaculis
6	5865	15-APR-26	Quentin Hoover	Hettie Sarver	Pediatrics	CANCELLED	ad rutrum vitae sollicitudin a Phasellus at dictum mollis porttitor dui eros Nulla
7	12238	15-APR-26	Leslie George	Nat Dornan	Orthopedics	NO_SHOW	ac taciti egestas fames Nunc ligula pede Proin per nunc est libero morbi inceptos Suspendiss
8	9802	15-APR-26	Wing Shields	Filberto Meeus	Cardiology	SCHEDULED	non mus rhoncus lacus pharetra ipsum faucibus neque conubia pretium fringilla volutpat Cras
9	1027	15-APR-26	Aristotle Nielsen	Leroi Gerrett	Cardiology	NO_SHOW	mauris tempor elementum Sed facilisi eleifend diam
10	28128	15-APR-26	Damian Obrien	Emilie McWhirter	Cardiology	COMPLETED	ac vitae eget auctor conubia faucibus nulla adipiscing taciti ultricies risus litora nec Pro
11	6946	15-APR-26	Anjolie Love	Ally Ashman	Pediatrics	SCHEDULED	Lorem scelerisque per imperdiet Maecenas aptent pulvinar viverra eget odio tortor Integer eg
12	48911	15-APR-26	Fleur Abbott	Phineas Liddicoat	General Surgery	COMPLETED	eget felis malesuada nascetur tortor Nunc vitae ultricies nonummy nostra Donec dui mi egesta
13	2366	15-APR-26	Patrick Gregory	Channa Shields	Emergency Medicine	NO_SHOW	eu tempor dolor sagittis sociis Sed venenatis facilisis nisi
14	32704	15-APR-26	Akeem Jacobs	Cecile Allmark	Cardiology	CANCELLED	erat feugiat gravida tincidunt lorem Vestibulum commodo Vestibulum tortor mus fames dis diam
15	40174	15-APR-26	Ariana Terry	Ally Ashman	Pediatrics	CANCELLED	Aenean nisl sed in dui elit nulla ad sit torquent eleifend Sed
16	24794	15-APR-26	Baxter Mccall	Trent Masden	Orthopedics	CANCELLED	auctor Curae purus venenatis Class pulvinar quis facilisi vitae vehicula torquent tincidunt
17	13027	15-APR-26	Zachary Palmer	Filberto Meeus	Cardiology	NO_SHOW	vel fringilla ante senectus parturient hymenaeos nisl primis In ligula
18	33455	15-APR-26	Jayme Barnes	Hilly Edowes	Cardiology	NO_SHOW	Sed elementum natoque sociis Integer molestie amet vitae facilisis Cras molestie laoreet nat

8.3.3 VW_DEPARTMENT_STATS

```
SELECT dept_name, total_doctors, total_patients, total_appointments
FROM vw_department_stats;
```

	DEPT_ID	DEPT_NAME	TOTAL_DOCTORS	TOTAL_PATIENTS	TOTAL_APPOINTMENTS
1	1	Cardiology	5	16802	8403
2	2	Neurology	4	13478	6678
3	3	Orthopedics	9	29797	15005
4	4	Pediatrics	3	10114	4966
5	5	Emergency	2	6606	3419
6	6	Oncology	2	6674	3317
7	7	General Surgery	5	16529	8212

8.3.4 VW_LOW_STOCK_MEDICATIONS

```
SELECT medication_id, med_name, category, stock_qty, unit
FROM vw_low_stock_medications;
```

MEDICATI...	MED_NAME	CATEGORY	STOCK_QTY	UNIT

8.4 Functions

8.4.1 FN_DOCTOR_APPT_COUNT

```
-- Count appointments for doctor 1 in 2024, month 3
SELECT fn_doctor_appt_count(1, 2024, 3) AS march_2024_appts FROM DUAL;

-- Try different doctor and month
SELECT fn_doctor_appt_count(2, 2023, 12) AS dec_2023_appts FROM DUAL;
```

	MARCH_2024_APPTS
1	11

	DEC_2023_APPTS
1	0

8.4.2 FN_PATIENT_AGE

```
SELECT fn_patient_age(DATE '1985-06-20') AS calculated_age FROM DUAL;

-- Use on actual patients
SELECT first_name, last_name, date_of_birth,
       fn_patient_age(date_of_birth) AS age
FROM patients WHERE ROWNUM <= 10;
```

	CALCULATED_AGE
1	40

	FIRST_NAME	LAST_NAME	DATE_OF_BIRTH	AGE
1	Beverly	Shepherd	08-JAN-72	54
2	Myles	Page	17-JUN-75	50
3	Imani	Rodriguez	16-MAR-95	31
4	Keane	Horn	31-DEC-62	63
5	Hadley	Oneill	28-NOV-99	26
6	Cody	Langley	07-DEC-86	39
7	Fredericka	Wise	13-FEB-56	70
8	Zachery	Velazquez	19-SEP-98	27
9	Patrick	Martinez	25-OCT-54	71
10	Ila	Cunningham	07-JUL-89	36

8.4.3 FN_LAST_DIAGNOSIS

```
SELECT fn_last_diagnosis(1) AS last_diagnosis FROM DUAL;

-- Show last diagnosis for first 10 patients
SELECT patient_id, fn_last_diagnosis(patient_id) AS last_diagnosis
FROM patients WHERE ROWNUM <= 10;
```

	PATIENT_ID	LAST_DIAGNOSIS
1	628	Hypertension
2	629	Fracture
3	630	Fracture
4	631	No records found
5	632	No records found
6	633	Hypertension
7	634	Migraine
8	635	No records found
9	636	No records found
10	637	Stroke

	LAST_DIAGNOSIS
1	Respiratory Infection

8.5 Procedures

8.5.1 SP_REGISTER_PATIENT

```
DECLARE
    v_new_id NUMBER;
BEGIN
    sp_register_patient(
        p_first_name      => 'John',
        p_last_name       => 'Doe',
        p_dob             => DATE '1980-11-25',
        p_gender          => 'M',
        p_blood_type      => 'O+',
        p_phone           => '0611223344',
        p_email           => 'john.doe@test.com',
        p_assigned_doctor => 2,
        p_new_id          => v_new_id
    );
    DBMS_OUTPUT.PUT_LINE('New patient ID: ' || v_new_id);
END;
/

-- Verify the new patient exists
SELECT patient_id, first_name, last_name, email
FROM patients WHERE email = 'john.doe@test.com';
```

```
New patient ID: 100003
```

	PATIENT_ID	FIRST_NAME	LAST_NAME	EMAIL
1	100003	John	Doe	john.doe@test.com

8.5.2 SP_UPDATE_SEVERITY

```
-- Check severity BEFORE procedure
SELECT record_id, diagnosis, severity
FROM medical_records
WHERE patient_id = 1 AND diagnosis = 'Hypertension'
  AND ROWNUM <= 5;

-- Run the procedure
EXEC sp_update_severity(1, 'Hypertension');

-- Check severity AFTER procedure
SELECT record_id, diagnosis, severity
FROM medical_records
WHERE patient_id = 1 AND diagnosis = 'Hypertension'
  AND ROWNUM <= 5;

-- Verify audit log entry was created
SELECT * FROM patient_audit_log WHERE patient_id = 1;
```

	RECORD_ID	DIAGNOSIS	SEVERITY
1	109620	Hypertension	MODERATE

	RECORD_ID	DIAGNOSIS	SEVERITY
1	109620	Hypertension	SEVERE

	LOG_ID	PATIENT_ID	CHANGED_BY	CHANGE_DATE	OLD_EMAIL	NEW_EMAIL	OLD_PHONE	NEW_PHONE
1	1	1	LQ0056	15-APR-26	updated.email@hospital.com	updated.email@hospital.com	998-440-6471	998-440-6471
2	2	1	LQ0056	15-APR-26	(null)	(null)	(null)	(null)
3	3	1	LQ0056	15-APR-26	(null)	(null)	(null)	(null)

8.6 Indexes and Execution Plans

8.6.1 Verify indexes exist

```
SELECT index_name, table_name, column_name
FROM user_ind_columns
WHERE table_name IN ('PATIENTS','APPOINTMENTS','MEDICAL_RECORDS','PRESCRIPTIONS')
ORDER BY table_name, index_name;
```

	INDEX_NAME	TABLE_NAME	COLUMN_NAME
1	IDX_APPTS_DATE	APPOINTMENTS	APPT_DATE
2	IDX_APPTS_DOCTOR_STATUS	APPOINTMENTS	DOCTOR_ID
3	IDX_APPTS_DOCTOR_STATUS	APPOINTMENTS	STATUS
4	IDX_APPTS_PATIENT	APPOINTMENTS	PATIENT_ID
5	SYS_C00459812	APPOINTMENTS	APPT_ID
6	IDX_RECORDS_PATIENT	MEDICAL_RECORDS	SYS_NC00010$
7	IDX_RECORDS_PATIENT	MEDICAL_RECORDS	PATIENT_ID
8	IDX_RECORDS_SEVERITY	MEDICAL_RECORDS	SEVERITY
9	SYS_IOT_TOP_1501829	MEDICAL_RECORDS	RECORD_ID
10	IDX_PATIENTS_DOB	PATIENTS	DATE_OF_BIRTH
11	IDX_PATIENTS_DOCTOR	PATIENTS	ASSIGNED_DOCTOR
12	IDX_PATIENTS_NAME	PATIENTS	LAST_NAME
13	IDX_PATIENTS_NAME	PATIENTS	FIRST_NAME
14	SYS_C00459823	PATIENTS	PATIENT_ID
15	PRESCRIPTIONS_PK	PRESCRIPTIONS	PRESCRIPTION_ID

8.6.2 Execution plan WITH index (index range scan)

```
EXPLAIN PLAN FOR
SELECT first_name, last_name FROM patients WHERE assigned_doctor = 1;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Look for: INDEX RANGE SCAN on IDX_PATIENTS_DOCTOR (fast)
```

```
3 ---------------------------------------------------------------------------------------------
4 | Id  | Operation                | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
5 ---------------------------------------------------------------------------------------------
6 |   0 | SELECT STATEMENT         |                       |  3366 | 60588 |   953   (1)| 00:00:01 |
7 |   1 |  VIEW                    | index$_join$_001      |  3366 | 60588 |   953   (1)| 00:00:01 |
8 |*  2 |   HASH JOIN              |                       |       |       |            |          |
9 |   3 |    INDEX FAST FULL SCAN| IDX_PATIENTS_NAME     |  3366 | 60588 |   506   (1)| 00:00:01 |
0 |*  4 |    INDEX FAST FULL SCAN| IDX_PATIENTS_DOCTOR   |  3366 | 60588 |   317   (1)| 00:00:01 |
1 ---------------------------------------------------------------------------------------------
```

8.6.3 Execution plan for medical records query

```
EXPLAIN PLAN FOR
SELECT patient_id, diagnosis, severity, visit_date
FROM medical_records
WHERE patient_id = 1
ORDER BY visit_date DESC;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Should show INDEX RANGE SCAN on IDX_RECORDS_PATIENT
```

```
PLAN_TABLE_OUTPUT
1 Plan hash value: 3283616478
2
3 ---------------------------------------------------------------------------------------------
4 | Id  | Operation          | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
5 ---------------------------------------------------------------------------------------------
6 |   0 | SELECT STATEMENT   |                       |     5 |  1430 |     8   (0)| 00:00:01 |
7 |*  1 |  INDEX UNIQUE SCAN| SYS_IOT_TOP_1501829   |     5 |  1430 |     8   (0)| 00:00:01 |
8 |*  2 |   INDEX RANGE SCAN| IDX_RECORDS_PATIENT   |     5 |       |     3   (0)| 00:00:01 |
9 ---------------------------------------------------------------------------------------------
10
11 Predicate Information (identified by operation id):
12 ---------------------------------------------------
13
14    1 - access("PATIENT_ID"=1)
15    2 - access("PATIENT_ID"=1)
16
17 Note
18 -----
19    - dynamic statistics used: dynamic sampling (level=2)
```

8.7 Partitions

```
-- List all partitions and their row counts
SELECT partition_name, num_rows, high_value
FROM user_tab_partitions
WHERE table_name = 'PATIENTS'
ORDER BY partition_position;

-- Query a specific partition directly
SELECT COUNT(*) AS patients_in_2023
FROM patients PARTITION (p_2023);

-- Show partition pruning in execution plan
EXPLAIN PLAN FOR
SELECT COUNT(*) FROM patients
WHERE admission_date BETWEEN DATE '2023-01-01' AND DATE '2023-12-31';
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Look for: PARTITION RANGE SINGLE (only one partition scanned)
```

	PARTITION_NAME	NUM_ROWS	HIGH_VALUE
1	P_BEFORE_2022	48577	TO_DATE(' 2022-01-01 00:00:00', 'SYYYY-MM-DD HH24:MI:SS', 'NLS_CALENDAR=GREGORIAN')
2	P_2022	12075	TO_DATE(' 2023-01-01 00:00:00', 'SYYYY-MM-DD HH24:MI:SS', 'NLS_CALENDAR=GREGORIAN')
3	P_2023	12117	TO_DATE(' 2024-01-01 00:00:00', 'SYYYY-MM-DD HH24:MI:SS', 'NLS_CALENDAR=GREGORIAN')
4	P_2024	12009	TO_DATE(' 2025-01-01 00:00:00', 'SYYYY-MM-DD HH24:MI:SS', 'NLS_CALENDAR=GREGORIAN')
5	P_2025_PLUS	15222	MAXVALUE

	PATIENTS_IN_2023
1	12117

	PLAN_TABLE_OUTPUT
1	Plan hash value: 4121220586
2	
3	--
4	| Id | Operation | Name | Rows | Bytes | Cost (%CPU)| Time | Pstart| Pstop |
5	--
6	| 0 | SELECT STATEMENT | | 1 | 8 | 274 (0)| 00:00:01 | | |
7	| 1 | SORT AGGREGATE | | 1 | 8 | | | | |
8	| 2 | PARTITION RANGE SINGLE| | 12117 | 96936 | 274 (0)| 00:00:01 | 3 | 3 |
9	|* 3 | TABLE ACCESS FULL | PATIENTS | 12117 | 96936 | 274 (0)| 00:00:01 | 3 | 3 |
10	--
11	
12	Predicate Information (identified by operation id):
13	---
14	
15	3 - filter("ADMISSION_DATE"<=TO_DATE(' 2023-12-31 00:00:00', 'syyyy-mm-dd hh24:mi:ss'))

8.8 Index-Organised Table (MEDICAL_RECORDS)

```
-- Confirm MEDICAL_RECORDS is an IOT
SELECT table_name, iot_type, row_movement
FROM user_tables
WHERE table_name = 'MEDICAL_RECORDS';

-- Query works just like a normal table
SELECT record_id, patient_id, diagnosis, severity
FROM medical_records
WHERE patient_id = 1
ORDER BY visit_date DESC
FETCH FIRST 10 ROWS ONLY;
```

	TABLE_NAME	IOT_TYPE	ROW_MOVEMENT
1	MEDICAL_RECORDS	IOT	DISABLED

	RECORD_ID	PATIE...	DIAGNOSIS	SEVERITY
1	140203	1	Respiratory Infection	MODERATE
2	81099	1	Migraine	MODERATE
3	83652	1	Type 2 Diabetes	MODERATE
4	10989	1	Respiratory Infection	MODERATE
5	109620	1	Hypertension	SEVERE

9. Privileges Granted to lkpeter

The following grants were issued to allow the professor (lkpeter) to query and use all database objects:

```
-- Tables
GRANT SELECT ON departments       TO lkpeter;
GRANT SELECT ON doctors           TO lkpeter;
GRANT SELECT ON patients          TO lkpeter;
GRANT SELECT ON appointments      TO lkpeter;
GRANT SELECT ON medications       TO lkpeter;
GRANT SELECT ON prescriptions     TO lkpeter;
GRANT SELECT ON medical_records   TO lkpeter;
GRANT SELECT ON patient_audit_log TO lkpeter;

-- Views
GRANT SELECT ON vw_patient_details       TO lkpeter;
GRANT SELECT ON vw_todays_appointments   TO lkpeter;
GRANT SELECT ON vw_department_stats      TO lkpeter;
GRANT SELECT ON vw_low_stock_medications TO lkpeter;
GRANT SELECT ON vw_prescription_history  TO lkpeter;

-- PL/SQL objects
GRANT EXECUTE ON fn_doctor_appt_count  TO lkpeter;
GRANT EXECUTE ON fn_patient_age        TO lkpeter;
GRANT EXECUTE ON fn_last_diagnosis     TO lkpeter;
GRANT EXECUTE ON sp_register_patient   TO lkpeter;
GRANT EXECUTE ON sp_update_severity    TO lkpeter;
GRANT EXECUTE ON sp_issue_prescription TO lkpeter;
```

10. Language Model Usage

As required by the language model usage policy, the following documents how AI assistance was used in this project.

Item	Details
Model name	Claude Sonnet 4.6 (claude-sonnet-4-6) by Anthropic
Version	claude-sonnet-4-6
Prompts used	1. Draft the technical documentation for an Oracle hospital database project including sections for Possible tools to use for test data documentation and generation, IOT, and PL/SQL logic. 2. How to fix ORA-04091 mutating table error in a compound trigger 3. How to fix appt_date format mismatch in SQL Developer import wizard 4. How to add partitioning and IOT after Data Modeler DDL export strips them. 5. Provide SQL examples to test and document the functionality of triggers, functions, and procedures.
How output was used	The generated text was used as a template. I manually corrected errors, updated specific schema details, and integrated my own SQL Developer screenshots to prove the objects worked. I also revised the PL/SQL code for sp_update_severity to handle multi-level upgrades based on my own logical requirements.
Validation method	All drafted documentation was verified by executing the associated SQL code in Oracle SQL Developer. Results were confirmed via the DBMS Output panel and the Explain Plan utility (F10) to ensure the descriptions matched the actual database performance.

11. Appendix — Full SQL Scripts

11.1 Schema Creation Order

- Step 1: Create sequences (all 7)
- Step 2: Create tables in FK dependency order
- Step 3: Add deferred FK (departments.head_doctor_id → doctors)
- Step 4: Create indexes
- Step 5: Create triggers
- Step 6: Create views
- Step 7: Create functions and procedures
- Step 8: Grant privileges to lkpeter

11.2 Tables Creation DDL

```
-- Generated by Oracle SQL Developer Data Modeler 24.3.1.347.1153
--   at:        2026-04-14 17:03:49 CEST
--   site:      Oracle Database 11g
--   type:      Oracle Database 11g



-- predefined type, no DDL - MDSYS.SDO_GEOMETRY

-- predefined type, no DDL - XMLTYPE

CREATE TABLE appointments (
    appt_id        NUMBER          PRIMARY KEY,
    patient_id     NUMBER          NOT NULL REFERENCES patients(patient_id),
    doctor_id      NUMBER          NOT NULL REFERENCES doctors(doctor_id),
    appt_date      DATE            NOT NULL,
    appt_duration  NUMBER          DEFAULT 30
                   CHECK (appt_duration IN (15, 30, 45, 60)),
    status         VARCHAR2(20)    DEFAULT 'SCHEDULED'
                   CHECK (status IN
('SCHEDULED','COMPLETED','CANCELLED','NO_SHOW')),
    reason         VARCHAR2(200),
    notes          VARCHAR2(500),
    created_at     DATE            DEFAULT SYSDATE
);

ALTER TABLE appointments
    ADD
        CHECK ( status IN ( 'CANCELLED', 'COMPLETED', 'NO_SHOW',
'SCHEDULED' ) );

ALTER TABLE appointments ADD CONSTRAINT appointments_pk PRIMARY KEY ( appt_id );

CREATE TABLE departments (
    dept_id        NUMBER NOT NULL,
    dept_name      VARCHAR2(50),
    location       VARCHAR2(50),
    head_doctor_id NUMBER
);

ALTER TABLE departments ADD CONSTRAINT departments_pk PRIMARY KEY ( dept_id );

CREATE TABLE doctors (
    doctor_id      NUMBER NOT NULL,
    first_name     VARCHAR2(50) NOT NULL,
    last_name      VARCHAR2(50) NOT NULL,
    specialization VARCHAR2(100),
    dept_id        NUMBER NOT NULL,
    license_number VARCHAR2(30) NOT NULL,
    hire_date      DATE DEFAULT sysdate,
    is_active      CHAR(1) DEFAULT 'Y'
);
```

```
ALTER TABLE doctors
    ADD CHECK ( is_active IN ( 'N', 'Y' ) );

ALTER TABLE doctors ADD CONSTRAINT doctors_pk PRIMARY KEY ( doctor_id );

ALTER TABLE doctors ADD CONSTRAINT doctors__un UNIQUE ( license_number );

CREATE TABLE medical_records (
    record_id   NUMBER        PRIMARY KEY,
    patient_id  NUMBER        NOT NULL REFERENCES patients(patient_id),
    doctor_id   NUMBER        NOT NULL REFERENCES doctors(doctor_id),
    visit_date  DATE          DEFAULT SYSDATE,
    diagnosis   VARCHAR2(500) NOT NULL,
    treatment   VARCHAR2(500),
    severity    VARCHAR2(20)  DEFAULT 'MODERATE'
                              CHECK (severity IN
('MILD','MODERATE','SEVERE','CRITICAL')),
    follow_up   DATE,
    notes       VARCHAR2(1000)
)
ORGANIZATION INDEX;

CREATE TABLE medications (
    medication_id NUMBER NOT NULL,
    med_name      VARCHAR2(150) NOT NULL,
    category      VARCHAR2(80),
    unit          VARCHAR2(20) DEFAULT 'mg',
    unit_price    NUMBER(10, 2),
    stock_qty     NUMBER DEFAULT 0,
    updated_at    DATE DEFAULT sysdate
);

ALTER TABLE medications ADD CHECK ( stock_qty > 0 );

ALTER TABLE medications ADD CONSTRAINT medications_pk PRIMARY KEY
( medication_id );

CREATE TABLE patients (
    patient_id       NUMBER          PRIMARY KEY,
    first_name       VARCHAR2(50)    NOT NULL,
    last_name        VARCHAR2(50)    NOT NULL,
    date_of_birth    DATE            NOT NULL,
    gender           CHAR(1)         CHECK (gender IN ('M','F','O')),
    blood_type       VARCHAR2(5),
    phone            VARCHAR2(20),
    email            VARCHAR2(100),
    address          VARCHAR2(255),
    admission_date   DATE            DEFAULT SYSDATE,
    assigned_doctor  NUMBER          REFERENCES doctors(doctor_id),
    is_active        CHAR(1)         DEFAULT 'Y' CHECK (is_active IN ('Y','N'))
)
PARTITION BY RANGE (admission_date) (
    PARTITION p_before_2022 VALUES LESS THAN (DATE '2022-01-01'),
    PARTITION p_2022        VALUES LESS THAN (DATE '2023-01-01'),
    PARTITION p_2023        VALUES LESS THAN (DATE '2024-01-01'),
    PARTITION p_2024        VALUES LESS THAN (DATE '2025-01-01'),
    PARTITION p_2025_plus   VALUES LESS THAN (MAXVALUE)
);

ALTER TABLE patients
    ADD
        CHECK ( gender IN ( 'F', 'M', 'O' ) );

ALTER TABLE patients
    ADD CHECK ( is_active IN ( 'N', 'Y' ) );

ALTER TABLE patients ADD CONSTRAINT patients_pk PRIMARY KEY ( patient_id );

CREATE TABLE prescriptions (
    prescription_id NUMBER NOT NULL,
    patient_id      NUMBER NOT NULL,
    doctor_id       NUMBER NOT NULL,
    medication_id   NUMBER NOT NULL,
    prescribed_date DATE,
    dosage_qty      NUMBER NOT NULL,
```

```
    duration_days   NUMBER NOT NULL,
    instructions    VARCHAR2(300)
);

ALTER TABLE prescriptions ADD CHECK ( dosage_qty > 0 );

ALTER TABLE prescriptions ADD CHECK ( duration_days > 0 );

ALTER TABLE prescriptions ADD CONSTRAINT prescriptions_pk PRIMARY KEY
( prescription_id );

ALTER TABLE appointments
    ADD CONSTRAINT appointments_doctors_fk FOREIGN KEY ( doctor_id )
        REFERENCES doctors ( doctor_id );

ALTER TABLE appointments
    ADD CONSTRAINT appointments_patients_fk FOREIGN KEY ( patient_id )
        REFERENCES patients ( patient_id );

ALTER TABLE doctors
    ADD CONSTRAINT doctors_departments_fk FOREIGN KEY ( dept_id )
        REFERENCES departments ( dept_id );

ALTER TABLE medical_records
    ADD CONSTRAINT medical_records_doctors_fk FOREIGN KEY ( doctor_id )
        REFERENCES doctors ( doctor_id );

ALTER TABLE medical_records
    ADD CONSTRAINT medical_records_patients_fk FOREIGN KEY ( patient_id )
        REFERENCES patients ( patient_id );

ALTER TABLE patients
    ADD CONSTRAINT patients_doctors_fk FOREIGN KEY ( assigned_doctor )
        REFERENCES doctors ( doctor_id );

ALTER TABLE prescriptions
    ADD CONSTRAINT prescriptions_doctors_fk FOREIGN KEY ( doctor_id )
        REFERENCES doctors ( doctor_id );

ALTER TABLE prescriptions
    ADD CONSTRAINT prescriptions_medications_fk FOREIGN KEY ( medication_id )
        REFERENCES medications ( medication_id );

ALTER TABLE prescriptions
    ADD CONSTRAINT prescriptions_patients_fk FOREIGN KEY ( patient_id )
        REFERENCES patients ( patient_id );

CREATE SEQUENCE appointments_appt_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER appointments_appt_id_trg BEFORE
    INSERT ON appointments
    FOR EACH ROW
    WHEN ( new.appt_id IS NULL )
BEGIN
    :new.appt_id := appointments_appt_id_seq.nextval;
END;
/

CREATE SEQUENCE departments_dept_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER departments_dept_id_trg BEFORE
    INSERT ON departments
    FOR EACH ROW
    WHEN ( new.dept_id IS NULL )
BEGIN
    :new.dept_id := departments_dept_id_seq.nextval;
END;
/

CREATE SEQUENCE doctors_doctor_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER doctors_doctor_id_trg BEFORE
    INSERT ON doctors
    FOR EACH ROW
    WHEN ( new.doctor_id IS NULL )
```

```
BEGIN
    :new.doctor_id := doctors_doctor_id_seq.nextval;
END;
/

CREATE SEQUENCE medical_records_record_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER medical_records_record_id_trg BEFORE
    INSERT ON medical_records
    FOR EACH ROW
    WHEN ( new.record_id IS NULL )
BEGIN
    :new.record_id := medical_records_record_id_seq.nextval;
END;
/

CREATE SEQUENCE medications_medication_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER medications_medication_id_trg BEFORE
    INSERT ON medications
    FOR EACH ROW
    WHEN ( new.medication_id IS NULL )
BEGIN
    :new.medication_id := medications_medication_id_seq.nextval;
END;
/

CREATE SEQUENCE patients_patient_id_seq START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER patients_patient_id_trg BEFORE
    INSERT ON patients
    FOR EACH ROW
    WHEN ( new.patient_id IS NULL )
BEGIN
    :new.patient_id := patients_patient_id_seq.nextval;
END;
/

CREATE SEQUENCE prescriptions_prescription_id START WITH 1 NOCACHE ORDER;

CREATE OR REPLACE TRIGGER prescriptions_prescription_id BEFORE
    INSERT ON prescriptions
    FOR EACH ROW
    WHEN ( new.prescription_id IS NULL )
BEGIN
    :new.prescription_id := prescriptions_prescription_id.nextval;
END;
/
```